Software Requirements Document (SRD)
Algorithmic Trading System

Document Control
Document Title: Software Requirements Document for Algorithmic Trading System
Version: 1.0
Date: April 24, 2025
Author: Senior Software Architect, Hedge Fund
Status: Draft
Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders
Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Software Architect	Initial draft of SRD for Algorithmic Trading System

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Lead Developer	[To be assigned]		

Table of Contents

1. Introduction

1.1. Purpose

The Software Requirements Document (SRD) for the Algorithmic Trading System (ATS) specifies the detailed software requirements necessary to design, develop, and deploy a high-performance platform for simulated trading. This document serves as a comprehensive guide for the development team, product owner, traders, and compliance officers to ensure alignment on the system's functionality, performance, and technical specifications. The SRD translates business and product requirements into actionable software requirements, detailing the system's behavior, interfaces, and constraints. It provides a foundation for system design, implementation, testing, and validation, ensuring the ATS meets the hedge fund's strategic objectives and regulatory standards.

1.2. Scope

The ATS is a sophisticated software platform designed to execute paper trades across multiple asset classes, including stocks, indices, forex, and commodities, on major global exchanges such as NYSE, NASDAQ, LSE, and NSE. Integrated with the Interactive Brokers (IBKR) Paper Trading API, the system leverages advanced data processing libraries (Polars, DuckDB) and AI agents to support algorithmic trading. The scope includes:

- **Trading Strategies**: Implementation of Trend-Following and Support and Resistance strategies for long and short positions.
- **Market Watch Dashboard**: A web-based, responsive interface for real-time market monitoring, technical analysis, portfolio management, and simulated trading.
- **Data Management**: Efficient ingestion, processing, storage, and visualization of market data.
- **Order Execution**: Placement and monitoring of simulated trades via the IBKR API.
- **Risk Management**: Position sizing, stop-loss, take-profit, and portfolio-level risk controls.
- **AI Agents**: Modular components for market analysis, trade execution, and risk assessment.
- **Reporting and Analytics**: Trade performance metrics, backtesting results, portfolio analytics, and compliance reports.
- **Security and Compliance**: Encrypted communications, secure credential management, and adherence to financial regulations.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Support for cryptocurrencies or non-traditional assets.
- Mobile app development (dashboard will be mobile-responsive).
- Advanced AI model training (pre-trained models will be used).
- Premium third-party data sources for initial phase.

1.3. Objectives

The ATS aims to achieve the following software objectives:

1. **Automate Trading Processes**: Implement algorithmic strategies to execute trades with minimal latency and high accuracy.
2. **Provide Real-Time Data Access**: Deliver live market data and technical indicators to support trading decisions.
3. **Ensure Robust Risk Management**: Enforce software-driven risk controls to protect capital and ensure sustainable trading.
4. **Support Strategy Validation**: Enable backtesting and paper trading to validate strategies without financial risk.
5. **Deliver Intuitive Interfaces**: Develop a responsive, user-friendly dashboard to enhance trader productivity.
6. **Meet Regulatory Standards**: Implement audit trails, logging, and reporting to comply with financial regulations.
7. **Ensure Scalability and Maintainability**: Design a modular, extensible architecture to support future enhancements.
8. **Optimize Performance**: Achieve low-latency data processing and trade execution to meet market demands.

1.4. Assumptions and Constraints

Assumptions:

- The IBKR Paper Trading API provides reliable, timely market data and order execution.
- Traders have intermediate knowledge of algorithmic trading and technical analysis.
- The Lenovo Legion Pro 5 (NVIDIA RTX 3060 GPU) supports system performance requirements.
- Regulatory requirements for paper trading are minimal compared to live trading.
- The development team is proficient in Python, Polars, DuckDB, React, and FastAPI.
- Budget and resources are sufficient for a 3-6 month development cycle.

Constraints:

- Development timeline: 3-6 months to align with strategic goals.
- IBKR API rate limits: 50 requests/second for market data, 100 messages/second for orders.
- Budget: No premium third-party data sources or cloud hosting initially.
- Deployment: Local on Lenovo Legion Pro 5, no cloud infrastructure.
- Compliance: Must adhere to SEC, GDPR, MiFID II, and SEBI regulations.
- Hardware: Limited to Lenovo Legion Pro 5 capabilities.

1.5. Stakeholders

Stakeholder	Role	Responsibilities
Head of Trading	Oversees trading operations	Defines strategy requirements, validates functionality
Traders	End-users	Test system, provide usability feedback
Product Owner	Represents business	Prioritizes features, approves deliverables
Development Team	Builds and tests system	Implements requirements, ensures quality
Compliance Officer	Ensures regulatory adherence	Reviews system for compliance
IT Administrator	Manages infrastructure	Configures hardware, ensures uptime
Risk Manager	Oversees risk controls	Defines risk parameters, monitors performance

1.6. Definitions and Acronyms

- **ATS**: Algorithmic Trading System.
- **IBKR**: Interactive Brokers.
- **OHLCV**: Open, High, Low, Close, Volume.
- **VW SMA**: Volume-Weighted Simple Moving Average.
- **MFI**: Money Flow Index.
- **ATR**: Average True Range.
- **Polars**: High-performance data processing library.
- **DuckDB**: In-memory analytical database.
- **FastAPI**: Python framework for APIs.
- **ib_insync**: Python library for IBKR API integration.
- **Gann Fan**: Technical analysis tool for dynamic support/resistance.
- **Fibonacci Retracement**: Percentage-based retracement levels.

---

2. System Overview

2.1. System Description

The Algorithmic Trading System (ATS) is a software platform for algorithmic paper trading, enabling simulated trades across stocks, indices, forex, and commodities. Integrated with the IBKR Paper Trading API, the ATS uses Polars and DuckDB for data processing and storage, and incorporates AI agents for enhanced decision-making. The system features two trading strategies (Trend-Following and Support and Resistance), a responsive Market Watch Dashboard, and robust risk management. Deployed locally on the Lenovo Legion Pro 5, the ATS employs a modular, event-driven architecture to ensure performance, scalability, and maintainability.

2.2. System Architecture

The ATS follows a microservices-based, event-driven architecture with the following components:

- **Data Layer**:
  - **Ingestion Service**: Fetches real-time and historical data via IBKR API using ib_insync.
  - **Processing Service**: Normalizes and computes indicators using Polars.
  - **Storage Service**: Stores time-series data in DuckDB with Parquet format.
  - **Cache**: Redis for low-latency data access.
- **Strategy Engine**:

    - Executes Trend-Following and Support and Resistance strategies.
    - Uses RabbitMQ for event-driven signal generation.
- **Order Execution Module**:
    - Manages trade lifecycle via IBKR API.
    - Handles order placement, monitoring, and error recovery.
- **Risk Management Module**:
    - Enforces position sizing, stop-loss, and portfolio risk controls.
    - Implements circuit breakers for anomaly detection.
- **AI Agents Framework**:
    - Modular agents for trade execution, risk management, market analysis, and API interaction.
    - Uses scikit-learn and TensorFlow for logic, RabbitMQ for communication.
- **Market Watch Dashboard**:
    - Frontend: React with Tailwind CSS and Plotly.js for visualization.
    - Backend: FastAPI for RESTful and WebSocket APIs.
- **Reporting Module**:
    - Generates trade, portfolio, and compliance reports.
    - Supports CSV, PDF, and PNG exports.
- **Monitoring and Logging**:
    - Prometheus and Grafana for metrics.
    - Structured logging with Python logging.

Architectural Principles:

- **Modularity**: Independent services for scalability and maintainability.
- **Event-Driven**: Asynchronous communication via RabbitMQ for low latency.
- **Separation of Concerns**: Isolates trading logic, risk management, and UI.
- **Security by Design**: Encryption and authentication at all layers.
- **Fault Tolerance**: Automatic reconnections and error handling.

2.3. Key Features

- **Multi-Asset Trading**: Supports stocks, indices, forex, and commodities.
- **Automated Strategies**: Executes Trend-Following and Support and Resistance strategies.
- **Real-Time Monitoring**: Displays live prices, volume, and indicators.
- **Interactive Dashboard**: Offers charts, portfolio summaries, and trading controls.
- **AI Insights**: Automates analysis, execution, and risk management.
- **Backtesting and Simulation**: Validates strategies in paper trading mode.
- **Security**: Ensures encrypted communications and secure credentials.
- **Reporting**: Provides performance, analytics, and compliance reports.

2.4. System Context

The ATS interacts with the following entities:

- **External Systems**:
    - IBKR Paper Trading API: Data and trade execution.
    - Future sources: News feeds, sentiment data (out of scope).
- **Internal Systems**:
    - Portfolio management tools for benchmarking.
    - Compliance systems for transaction logging.
- **Users**:
    - Traders: Interact via dashboard.
    - Administrators: Manage configurations.
- **Environment**:
    - Local deployment on Lenovo Legion Pro 5.
    - Dockerized services for consistency.

---

3. Functional Requirements

3.1. Trading Strategies

The ATS implements two positional trading strategies for long and short positions in the IBKR paper trading environment.

3.1.1. Trend-Following Strategy

Description: Captures long-term price trends using volume-weighted indicators.
Requirements:

- **Assets**: Stocks (NYSE, NASDAQ, LSE, NSE), indices (S&P 500, FTSE 100, NIFTY 50), forex (EUR/USD, USD/JPY), commodities (gold, crude oil).
- **Technical Indicators**:
  - 55-day VW SMA of Open:
    $$\text{VW SMA} = \frac{\sum (\text{Open Price}_i \times \text{Volume}_i)}{\sum \text{Volume}_i}$$
  - 13-day VW SMA of Open (optional).
  - 14-day VW MFI of HLC Average:
    - HLC Average = (High + Low + Close) / 3.
    - Raw Money Flow = HLC Average × Volume.
    - MFI = 100 - (100 / (1 + (Positive Money Flow / Negative Money Flow))).
  - 34-day and 21-day VW SMA of MFI(14).
  - 5-day VW SMA of High/Low.
  - 21-day VW ATR:
    - True Range = max(High - Low, |High - Previous Close|, |Low - Previous Close|).
    - ATR = VW SMA of True Range.
- **Entry Rules**:
  - Long: MFI(14) > 34-day VW SMA of MFI(14) and price > 55-day VW SMA of Open.
  - Short: MFI(14) < 34-day VW SMA of MFI(14) and price < 55-day VW SMA of Open.
- **Exit Rules**:
  - Long:
    - Stop-Loss: Entry - (2 × 21-day VW ATR).
    - Take-Profit: Entry + (2 × 21-day VW ATR).
    - Additional: MFI(14) < 21-day VW SMA of MFI(14) or price < 5-day VW SMA of Low.
  - Short:
    - Stop-Loss: Entry + (2 × 21-day VW ATR).
    - Take-Profit: Entry - (2 × 21-day VW ATR).
    - Additional: MFI(14) > 21-day VW SMA of MFI(14) or price > 5-day VW SMA of High.
- **Position Sizing**:
  - Risk: 1% of £10,000 account.
  - Formula:
    $$\text{Position Size} = \frac{\text{Account Balance} \times 0.01}{2 \times \text{21-day VW ATR} \times \text{Lot Size}}$$
  - Lot Size: Stocks (1 share), forex (100,000 units), commodities (per contract), indices (per multiplier).
- **Money Management**:
  - Drawdown > 20%: Reduce risk to 0.5%.
  - Trailing Stop: 1.5 × 21-day VW ATR.
  - Convert to GBP using IBKR rates.
- **Implementation**:
  - Polars for indicator calculations.
  - RabbitMQ for event-driven signals.
  - Log signals for auditing.

3.1.2. Support and Resistance Strategy
Description: Trades breakouts/reversals at support/resistance levels.
Requirements:

- **Assets**: Same as Trend-Following.

- **Technical Tools**:
  - Pivot Points:
    - PP = (High + Low + Close) / 3.
    - R1 = 2 × PP - Low.
    - S1 = 2 × PP - High.
    - R2 = PP + (High - Low).
    - S2 = PP - (High - Low).
  - Fibonacci Retracement: 23.6%, 38.2%, 50%, 61.8%, 100%.
  - Gann Fan: 1x1, 1x2, 2x1 angles.
- **Entry Rules**:
  - Long: Price breaks R1/Fibonacci (e.g., 61.8%) with volume; or bounces off Gann Fan angle in uptrend.
  - Short: Price breaks S1/Fibonacci (e.g., 38.2%) with volume; or rejected at Gann Fan angle in downtrend.
  - Optional: Candlestick confirmation (e.g., bullish engulfing).
- **Exit Rules**:
  - Long:
    - Take-Profit: R2, next Fibonacci, or Gann Fan angle.
    - Stop-Loss: Below S1, swing low, or 1-2%.
  - Short:
    - Take-Profit: S2, next Fibonacci, or Gann Fan angle.
    - Stop-Loss: Above R1, swing high, or 1-2%.
  - Trailing Stop: 2-3% or 2 × 14-day ATR.
- **Position Sizing**:
  - Risk: 1-2%.
  - Formula:
    $$\text{Position Size} = \frac{\text{Account Balance} \times \text{Risk %}}{\text{|Entry Price - Stop Loss Price|}}$$
- **Money Management**:
  - Portfolio risk: <5-10%.
  - Diversify assets/sectors.
  - Adjust using 14-day ATR.
- **Implementation**:
  - Polars for calculations.
  - Plotly.js for Gann Fan visualization.

3.2. Market Watch Dashboard

Description: Web-based interface for market monitoring and trading.

3.2.1. Scrip Selection

- Dropdown/searchable input for tickers.
- Support multiple exchanges.
- Auto-suggest; validate via IBKR API.
- Cache recent selections.

3.2.2. Real-Time Data Display

- Show bid, ask, last price, volume, OHLCV.
- Update every 5-10s via WebSocket.
- Highlight price changes.
- Fields: Ticker, exchange, bid, ask, last, volume, % change.

3.2.3. Interactive Charts

- Candlestick/line charts with zoom/pan.
- Overlay: VW SMA, MFI, ATR, RSI(14), MACD, Pivot Points, Fibonacci, Gann Fan.
- Timeframes: 1m, 5m, 15m, 1h, 4h, 1d, 1w.
- Crosshair, PNG export, save settings.

3.2.4. Portfolio Summary

- Show positions, account value, P&L.
- Fields: Ticker, size, entry price, current price, P&L, period.
- Filter by asset, exchange, strategy.

3.2.5. Simulated Trading Panel

- Place buy/sell (market, limit, stop, bracket).
- Input: Ticker, quantity, price, order type.
- Validate; show status.
- Preview impact; cancel/modify orders.

3.2.6. Watchlist

- Save scrips.
- Highlight volatility/volume.
- Reorder/group by asset.

3.2.7. Technical Analysis

- Outlook: Strong Uptrend, Uptrend, Sideways, Downtrend, Strong Downtrend.
- Indicators: RSI(14), MFI(14), MACD, VW SMA, EMA.
- Support/resistance with AI recommendations.

3.2.8. News and Events

- Show scrip news/events (AGM, dividends) within 10 days.
- Filter by relevance/date.
- Placeholder for API integration.

3.3. Data Management

Description: Manages market data for trading.

3.3.1. Data Ingestion

- Fetch OHLCV via IBKR WebSocket.
- Historical data (5 years) for backtesting.
- Poll every 5-10s; cache in Redis.

3.3.2. Data Processing

- Normalize timestamps (UTC), asset IDs.
- Clean: Remove outliers, interpolate missing values.
- Compute indicators with Polars.

3.3.3. Data Storage

- DuckDB with Parquet, partitioned by date/asset.
- Retain 7 years.
- Daily backups.

3.3.4. Data Visualization

- Render price/indicator/performance charts.
- Multi-timeframe analysis.
- Export PNG/CSV.

3.4. Order Execution

Description: Manages trade lifecycle.

3.4.1. Order Types

- Market, limit, stop, bracket.
- Support partial fills.

3.4.2. Order Placement

- Submit via ib_insync with validation.
- Calculate size; confirm submission.

3.4.3. Order Monitoring

- Track status (pending, filled, canceled).
- Update portfolio/P&L; notify users.

3.4.4. Error Handling

- Handle API errors; retry 3 times.
- Log errors; alert critical failures.

3.5. Risk Management

Description: Enforces capital protection.

3.5.1. Position Sizing
- Risk: 1-2%.
- Trend-Following: 21-day VW ATR.
- Support and Resistance: Stop-loss distance.

3.5.2. Stop-Loss and Take-Profit
- Trend-Following: 2 × 21-day VW ATR.
- Support and Resistance: S1/R1 or 1-2%.
- Place with IBKR API.

3.5.3. Trailing Stop
- Trend-Following: 1.5 × 21-day VW ATR.
- Support and Resistance: 2-3% or 2 × 14-day ATR.

3.5.4. Portfolio Risk
- Open risk: <5-10%.
- Monitor correlations; diversify.

3.5.5. Drawdown Limits
- Drawdown > 20%: Risk to 0.5%.
- Drawdown > 30%: Halt trading.

3.5.6. Circuit Breakers
- Halt if price change > 5% in 5min.
- Pause for anomalies; resume after approval.

3.5.7. Currency Handling
- Convert to GBP using IBKR rates.
- Cache rates; log conversions.

3.6. AI Agents Integration
Description: Automates tasks with oversight.

3.6.1. Trade Execution Agent
- Execute trades based on signals.
- Optimize timing/price.

3.6.2. Risk Management Agent
- Enforce sizing, exposure, circuit breakers.
- Adjust risk dynamically.

3.6.3. Market Analysis Agent
- Analyze trends, volatility, momentum.
- Generate recommendations.

3.6.4. IBKR API Agent
- Manage API interactions.
- Optimize calls; handle errors.

3.6.5. Transparency and Oversight
- Log decisions.
- Require human approval for critical actions.
- Allow pausing/overriding.

3.7. Reporting and Analytics
Description: Generates reports.

3.7.1. Trade Performance
- Metrics: P&L, win rate, Sharpe ratio, max drawdown.
- Visualize equity curves.

3.7.2. Portfolio Analytics
- Exposure, correlations, VaR.
- Diversification analysis.

3.7.3. Backtesting Reports
- Compare vs. benchmarks.
- Parameter sensitivity.

3.7.4. Compliance Reports
- Transaction logs with timestamps/user IDs.

- Immutable audit trails.

3.7.5. Export Capabilities

- Export CSV, PDF, PNG.
- Schedule automated reports.

---

4. Non-Functional Requirements

4.1. Performance

- Latency: <100ms end-to-end.
- Throughput: 10,000 updates/s, 1,000 trades/min.
- Indicator calculation: <1s for 100 scrips.
- Dashboard refresh: Every 5-10s.
- Backtesting: 5 years data in <10min.

4.2. Scalability

- Vertical: Use Lenovo Legion Pro 5.
- Horizontal: Microservices for cloud.
- Data: 1TB historical data.
- Users: 10 concurrent.

4.3. Security

- Authentication: OAuth2, JWT, MFA.
- Data: TLS 1.3, AES-256.
- Credentials: Encrypted vault.
- API: Rate limiting, CSRF tokens.
- Agents: Sandboxed.

4.4. Reliability and Fault Tolerance

- Uptime: 99.9%.
- Error Handling: Auto-reconnect.
- Data Integrity: Validate data.
- Circuit Breakers: Halt anomalies.
- Backup: Daily, RTO <1h.

4.5. Usability

- Navigation: <5min training.
- UI Response: <1s.
- Accessibility: WCAG 2.1.
- Documentation: Confluence.

4.6. Maintainability

- Code Coverage: >80%.
- Modularity: Independent services.
- Logging: Structured, queryable.
- Documentation: API specs, guides.

4.7. Compliance

- Adhere to SEC, GDPR, MiFID II, SEBI.
- Log trades/orders immutably.
- Retain logs 7 years.
- Monitor market abuse.

---

5. Integration Requirements

5.1. Interactive Brokers Paper Trading API

- Connection: Host: 127.0.0.1, Port: 7497, Client ID: 0, Account: DUK221396.
- Data: Real-time/historical OHLCV.
- Orders: Market, limit, stop, bracket.
- Rate Limits: 50 req/s (data), 100 msg/s (orders).
- Security: Encrypted credentials.

5.2. External Data Sources

- Placeholder for news (Reuters, Bloomberg).

- Future sentiment/economic data.
- REST/WebSocket for real-time.

5.3. Internal Systems

- Sync with portfolio tools.
- Export logs to compliance systems.
- Store in warehouse via REST.

---

6. System Constraints

6.1. Hardware Constraints

- Lenovo Legion Pro 5 (NVIDIA RTX 3060).
- Limited to local deployment.
- Optimize for GPU/CPU usage.

6.2. Software Constraints

- Python 3.10, React 18.
- No premium data sources.
- Docker for containerization.

6.3. Regulatory Constraints

- Comply with SEC, GDPR, MiFID II, SEBI.
- Immutable logs for 7 years.
- Monitor market abuse.

---

7. Assumptions and Dependencies

7.1. Assumptions Log

ID	Description	Impact if Invalid	Validation Plan
A001	IBKR API reliable	Delayed trades	Test connectivity
A002	Traders know technical analysis	Increased training	Conduct training
A003	Hardware sufficient	Performance issues	Benchmark hardware
A004	Minimal paper trading regulations	Compliance delays	Consult Compliance Officer

7.2. Dependencies

- IBKR API availability.
- Polars, DuckDB, ib_insync.
- Lenovo Legion Pro 5.
- Team expertise.

---

8. Missing Information and Best Practices

8.1. Missing Information

- Exact VW indicator formulas.
- Detailed trader personas.
- Specific regulatory requirements.
- Performance benchmarks.
- News feed providers.

8.2. Best Practices Applied

- **Indicators**: Standard VW formulas. **Reason**: Industry-standard, ensures accuracy.
- **Personas**: Assumed intermediate traders. **Reason**: Typical for hedge funds.
- **Compliance**: Audit trails. **Reason**: Prepares for live trading.
- **Performance**: <100ms latency. **Reason**: High-frequency trading standard.
- **News**: Planned Reuters/Bloomberg. **Reason**: Reliable sources.

---

9. Implementation Considerations

9.1. Development Methodology

- Agile Scrum: 2-week sprints.
- TDD: Write tests first.
- CI/CD: GitHub Actions.

9.2. Technology Stack

- Backend: Python 3.10, FastAPI, ib_insync.
- Data: Polars, DuckDB.
- Frontend: React 18, Tailwind CSS, Plotly.js.
- Infrastructure: Docker, GitHub Actions.
- Monitoring: Prometheus, Grafana.

9.3. Testing Strategy

- Unit: Validate components with pytest.
- Integration: Test module interactions.
- System: End-to-end workflows.
- Performance: Measure latency with Locust.
- Security: OWASP ZAP scans.
- Backtesting: Historical data validation.
- Coverage: >80%.

9.4. Deployment Strategy

- Local on Lenovo Legion Pro 5.
- Docker Compose for services.
- CI/CD: Auto-deploy to test, manual to prod.
- Monitoring: Prometheus/Grafana.
- Rollback: Automated for failures.

---

10. Glossary

- **ATS**: Algorithmic Trading System.
- **IBKR**: Interactive Brokers.
- **VW SMA**: Volume-Weighted Simple Moving Average.
- **MFI**: Money Flow Index.
- **ATR**: Average True Range.
- **Polars**: Data processing library.
- **DuckDB**: Analytical database.

---

11. Appendices

11.1. Data Models

OHLCV:

```
{
 "ticker": "AAPL",
 "exchange": "NASDAQ",
 "timestamp": "2025-04-24T10:00:00Z",
 "open": 150.25,
 "high": 151.10,
 "low": 149.80,
 "close": 150.90,
 "volume": 1000000
}
```

Trade Log:

```
{
 "trade_id": "T12345",
 "timestamp": "2025-04-24T10:01:00Z",
 "ticker": "AAPL",
 "strategy": "Trend-Following",
 "action": "BUY",
 "quantity": 100,
 "price": 150.90,
 "status": "FILLED"
}
```

11.2. API Specifications

GET /market-data:

- Parameters: ticker, timeframe.
- Response: OHLCV data.
- Example: GET /market-data?ticker=AAPL&timeframe=1d.

POST /orders:

- Body: { ticker, action, quantity, price, type }.
- Response: Order ID, status.

11.3. Mockups and Wireframes

- Dashboard: Grid with scrip selector, chart, portfolio, trading panel.
- Chart: Candlestick with indicators.
- Trading Panel: Form with inputs.

11.4. References

- IBKR API: https://interactivebrokers.github.io/tws-api/
- Polars: https://pola.rs/
- DuckDB: https://duckdb.org/
- FastAPI: https://fastapi.tiangolo.com/
- React: https://reactjs.org/